POLYTECH NANTES | Pôle Sciences et technologie Nantes Université

Rapport de Languages matériels et réutilisation

ETN5 - SETR

Élèves :

Nolan BUCHET

Maxence BOULEAU

Date :

Octobre 2026

Table des matières

1 Introduction **2**

2 Récupération d'un modèle de RAM **3**
2.1 Validation du fonctionnement de la RAM 3
2.2 Modification du code de la RAM 3
2.3 Ajout de la généricité de la RAM 3
2.4 Ajout du Chip Select 4

3 Création du Bloc QuadRAM **5**
3.1 Conception et validation du décodeur d'adresses 5
3.2 Instanciation de 4 blocs SyncRam 5
3.3 Généralisation de l'architecture 6
3.4 Validation de la QuadRAM 6

4 Prise en main d'un modèle de processeur **7**
4.1 RISC-V RV32I et chargement des instructions 7
4.2 Analyse des modes d'adressage et extension RV32IM 7
4.2.1 Modes d'adressage 7
4.2.2 Évolution vers le jeu d'instructions RV32IM 8
4.3 Passage du jeu d'instructions RV32I au jeu RV32IM 8

5 Intégration Système : Processeur et Mémoire **9**
5.1 Modification du processeur 9
5.2 Création d'un bloc Top-Level 9
5.3 Validation du système intégral 10

1 Introduction

Ce mini-projet vise à concevoir et valider par simulation un système numérique sur puce articulé autour d'un processeur RISC-V et d'une mémoire modulaire, tout en mettant en application les principes de généricité et de réutilisation de blocs matériels (IP cores). La démarche consiste d'abord à transformer une mémoire RAM élémentaire en VHDL pour la rendre paramétrable et à instancier un bloc mémoire étendu (quadram) associé à un décodeur d'adresses. Dans un second temps, le modèle de processeur décrit en Verilog est adapté pour supporter l'extension d'instructions RV32IM et pour s'interfacer avec la mémoire externe. L'ensemble est enfin réuni au sein d'une entité englobante mixte VHDL/Verilog puis validé sous ModelSim à l'aide de programmes de test en assembleur, illustrant ainsi l'intégralité du flot de co-conception et d'intégration système.

2 Récupération d'un modèle de RAM

2.1 Validation du fonctionnement de la RAM

Nous récupérons le modèle `ramchip.vhd` et son testbench afin de pouvoir observer et analyser son comportement sur ModelSim.



FIGURE 1 – Chronogramme de la RAM

Dans le chronogramme ci-dessus, nous observons que notre simulation représente correctement le comportement d'une RAM.

Le bon fonctionnement se valide en commençant par mettre `WE` à 1. Sans celui-ci, la RAM ne peut pas écrire de données. Ensuite, à chaque front d'horloge, nous mettons à jours une addresse dans laquelle nous voulons écrire une donnée. Nous mettons ensuite à jour la donnée que nous voulons écrire dans la RAM.

— A 20 ns : Ecriture de 01 à l'addresse 001 -> DataOut = 01
— A 30 ns : Ecriture de 02 à l'addresse 002 -> DataOut = 02
— A 40 ns : Ecriture de 03 à l'addresse 003 -> DataOut = 03, et ainsi de suite

Un effet de décalage apparait, la valeur affichée correspond toujours à la donnée appliquée au cycle précédent. Exemple entre 20 ns et 30 ns, Address vaut 001, DataIn vaut 01, et DataOut restitue 00. Celui ci est normal car notre Ram est synchrone, et ne restitue donc la donnée qu'au cycle suivant.

2.2 Modification du code de la RAM

Dans un second temps, nous mettons à jour le code en remplaçant les `std_logic_vector` par des `unsigned` afin de pouvoir effectuer des opérations arithmétiques sur les adresses. Le fonctionnement reste identique à celui de la RAM initiale.

2.3 Ajout de la généricité de la RAM

Pour rendre notre RAM plus générique, nous ajoutons des paramètres de taille de données et d'adresses. Nous ajoutons donc les généricités `DATA_LENGTH` et `ADDR_LENGTH` qui permettront de définir la taille de la RAM.

```
1 entity sync_ram is
2   generic (
```

```
    ADDR_LENGHT : integer := 10;
    DATA_LENGHT : integer := 8
  );
  port (
    clock   : in  std_logic;
    we      : in  std_logic;
    address : in  unsigned((ADDR_LENGHT - 1) downto 0);
    datain  : in  unsigned((DATA_LENGHT - 1) downto 0);
    dataout : out unsigned((DATA_LENGHT - 1) downto 0)
  );
end entity sync_ram;
```

Listing 1 – Nouvelle entité de la RAM

2.4 Ajout du Chip Select

L'ajout du signal asynchrone `cs` permet de contrôler l'accès à la RAM et de placer ses sorties en haute impédance ('Z') en cas d'inactivité, en vue de la création du `QuadRAM`.

```
entity sync_ram is
  generic(DataLength : INTEGER := 8; AddrLength : INTEGER := 8);
  port (
    cs    : in std_logic;
    clock : in std_logic;
    ...
  );
end entity;

architecture RTL of sync_ram is
  ...
begin
  process(clock) is
  begin
    if rising_edge(clock) and cs = '1' then
      -- Ecriture synchronisee
    end if;
  end process;

  dataout <= ram(to_integer(unsigned(read_address))) when cs = '1
    ' else (others => 'Z');
end architecture;
```

Listing 2 – Ajout du signal CS

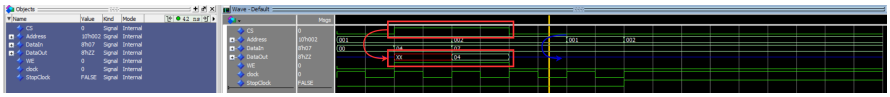


FIGURE 2 – Chronogramme de la RAM avec le signal CS

3 Création du Bloc QuadRAM

3.1 Conception et validation du décodeur d'adresses

Un décodeur d'adresses générique a été conçu pour piloter les signaux `cs` des blocs RAM à partir des bits de poids fort de l'adresse. Son fonctionnement a été validé par simulation (Figure 3).

```
entity decoder is
  generic(SEL_LENGTH : INTEGER := 2);
  port (
    address : in  unsigned(SEL_LENGTH - 1 downto 0);
    cs      : out unsigned((2**SEL_LENGTH) - 1 downto 0)
  );
end entity;

architecture RTL of decoder is
begin
  process(address) is
  begin
    cs <= (others => '0');
    cs(to_integer(address)) <= '1';
  end process;
end architecture;
```

Listing 3 – Décodeur d'adresses générique

FIGURE 3 – Chronogramme du décodeur d'adresses

3.2 Instanciation de 4 blocs SyncRam

L'entité `QuadRAM` intègre un décodeur d'adresses et une boucle `for...generate` pour instancier les blocs mémoire. Le décodeur analyse les bits de poids fort de l'adresse pour piloter individuellement les signaux `cs` de chaque RAM, tandis que les autres signaux (horloge, écriture et données) sont mutualisés.

```
DECODER : entity work.decoder(RTL)
generic map (
    SEL_LENGTH => 2
)
port map (
    cs      => cs_array,
    address => address(9 downto 8)
);

GenRam : for i in 0 to (2**2) - 1 generate
    Ram : entity work.sync_ram(RTL)
    generic map (
        DataLength => 8,
        AddrLength => 8
```

```
    )
    port map (
        cs      => cs_array(i),
        clock   => clock,
        we      => we,
        address => address(7 downto 0),
        datain  => datain,
        dataout => dataout
    );
end generate;
```

Listing 4 – QuadRAM

3.3 Généralisation de l'architecture

Pour généraliser l'architecture de la RAM, nous avons ajouté des généricités pour la taille des données et des adresses. Cela permet de définir la taille de la RAM en fonction des besoins spécifiques du projet.

```
  generic(ADDR_SELECT : INTEGER := 2;
          ADDR_LENGHT : INTEGER := 10;
          DATA_LENGHT : INTEGER := 8
  );
```

Listing 5 – QuadRAM Généralisée

3.4 Validation de la QuadRAM

Afin de valider le fonctionnement de la QuadRAM, nous avons réalisé une simulation en utilisant un test bench. Le chronogramme obtenu (Figure 4) montre que la QuadRAM fonctionne correctement, avec les signaux `cs` activés pour chaque bloc RAM en fonction des bits de poids fort de l'adresse.

FIGURE 4 – Chronogramme de la QuadRAM

4 Prise en main d'un modèle de processeur

4.1 RISC-V RV32I et chargement des instructions

Dans un premier temps, nous avons étudié le jeu d'instructions RISC-V RV32I et chargé les instructions dans la mémoire du processeur. Un *testbench* a été créé afin de simuler le processeur et de vérifier le bon fonctionnement des instructions.

L'exemple ci-dessous illustre le chargement d'un programme de test :

```
// instructions at address 0x0
@0
00200093 // addi x1, x0, 2
00400113 // addi x2, x0, 4
00110133 // add x2, x2, x1
001150b3 // srl x1, x2, x1
0420a023 // sw x2, 64(x1)
00100073 // ebreak
```

Listing 6 – Exemple de chargement d'instructions

Le chronogramme ci-dessous (Figure **??**) met en évidence l'exécution séquentielle de ces instructions :

- Le registre `x1` est initialisé à 2, puis `x2` à 4.
- Leur addition est stockée dans `x2`.
- Un décalage logique à droite de `x2` par `x1` est affecté à `x1`.
- Enfin, la valeur de `x2` est sauvegardée en mémoire à l'adresse calculée par l'offset 64 et la valeur de `x1`.

Les figures suivantes détaillent respectivement le comportement interne des registres (Figure 5) et l'écriture effective des données dans l'espace mémoire.

FIGURE 5 – Détail des opérations dans les registres

FIGURE 6 – Validation du stockage de la valeur en mémoire

4.2 Analyse des modes d'adressage et extension RV32IM

4.2.1 Modes d'adressage

L'architecture RISC-V manipule différents modes d'adressage pour accéder aux données :

- **Adressage immédiat :** La valeur de la donnée est directement encodée au sein même de l'instruction (ex : `addi x1, x0, 2`).
- **Adressage direct :** L'adresse de la mémoire ciblée est explicitement fournie dans l'instruction.
- **Adressage indirect (basé sur un registre) :** L'adresse mémoire effective est contenue dans un registre, souvent combinée avec un décalage constant (*offset*), comme dans l'instruction de stockage `sw x2, 64(x1)`[cite : 3].

4.2.2 Évolution vers le jeu d'instructions RV32IM

Le processeur fourni gère nativement le jeu d'instructions de base RV32I. Pour le rendre compatible avec l'extension **M** (*Multiplication and Division*), les modifications matérielles suivantes sont requises :

- Intégration d'unités de calcul arithmétique dédiées dans l'ALU pour la multiplication et la division entières (instructions `mul`, `div`).
- Décodage des nouveaux *opcodes* et *funct3/funct7* spécifiques à l'extension M dans l'unité de contrôle.
- Adaptation du programme de test (‘instr.mem‘) pour valider ces nouvelles opérations arithmétiques.

4.3 Passage du jeu d'instructions RV32I au jeu RV32IM

Voici les modifications apportées au code Verilog du processeur pour supporter l'extension M :

```
// Add functions RV32IM
‘define MUL    {3’b000, 7’b0000001}
‘define DIV    {3’b100, 7’b0000001}

// Addictionnal function
‘MUL: regs[‘RD] <= regs[‘RS1] * regs[‘RS2];
‘DIV: regs[‘RD] <= regs[‘RS1] / regs[‘RS2];
```

Listing 7 – Ajout de l'extension M dans le processeur

Nous modifions également le fichier ‘instr.mem‘ pour inclure des instructions :

```
// div
00200093 // addi x1, x0, 2
00400113 // addi x2, x0, 4
02114133 // div x2, x2, x1

// mul
00200093 // addi x1, x0, 2
00400113 // addi x2, x0, 4
02110133 // mul x2, x2, x1
```

Listing 8 – Instructions pour l'extension M

Voici le chronogramme résultant de l'exécution de ces instructions (Figure 10) :

FIGURE 7 – Chronogramme du processeur pour multiplication

FIGURE 8 – Chronogramme du processeur pour division

5 Intégration Système : Processeur et Mémoire

5.1 Modification du processeur

Le processeur a été modifié pour déporter son stockage de données vers la mémoire externe `QuadRAM`, en intégrant les bus d'adresses, de données et les signaux de contrôle nécessaires (`we`, `clk`, `rst`)[cite : 5].

```
module system (
  input wire clk, rst,
  output reg [11:0] mem_addr,
  output reg [31:0] mem_wdata,
  input  wire [31:0] mem_rdata,
  output reg         mem_we
);
```

Listing 9 – Interface externe du processeur

5.2 Création d'un bloc Top-Level

Un composant *Top-Level* rédigé en VHDL relie le processeur (en Verilog) et le bloc `QuadRAM`, en assurant la conversion des types entre les signaux standards et les vecteurs non signés.

```
entity system_top is
    Port (
        clk : in STD_LOGIC;
        rst : in STD_LOGIC
    );
end entity system_top;

architecture RTL of system_top is

    signal mem_addr  : std_logic_vector(11 downto 0);
    signal mem_wdata : std_logic_vector(31 downto 0);
    signal mem_rdata : std_logic_vector(31 downto 0);
    signal mem_we    : std_logic;

    signal quadram_addr  : unsigned(11 downto 0);
    signal quadram_wdata : unsigned(31 downto 0);
    signal quadram_rdata : unsigned(31 downto 0);

begin

    quadram_addr  <= unsigned(mem_addr);
    quadram_wdata <= unsigned(mem_wdata);
    mem_rdata     <= std_logic_vector(quadram_rdata);

    processor_inst : entity work.system
        port map (
            clk       => clk,
            rst       => rst,
```

```
29              mem_addr  => mem_addr,
30              mem_wdata => mem_wdata,
31              mem_rdata => mem_rdata,
32              mem_we    => mem_we
33          );
34
35      quadram_inst : entity work.quadram
36          generic map (
37              ADDR_SELECT => 2,
38              ADDR_LENGHT => 12,
39              DATA_LENGHT => 32
40          )
41          port map (
42              clock    => clk,
43              we       => mem_we,
44              address  => quadram_addr,
45              datain   => quadram_wdata,
46              dataout  => quadram_rdata
47          );
48
49  end architecture RTL;
```

Listing 10 – Extrait du Top-Level système

5.3 Validation du système intégral

Dans le même esprit que pour la validation du modèle processeur, nous validons notre système intégral en simulant l'exécution d'un programme de test. Le chronogramme ci-dessous (Figure **??**) illustre le fonctionnement du processeur en interaction avec la mémoire `QuadRAM`.

- Réalisation d'une addition (X1 = 2 ; X2 = 4 ; X2 = X1 + X2 ;)
- Stockage dans la mémoire (X2 = 6 ; @addr = 0 ;)
- Réalisation d'une multiplication (X1 = 2 ; X2 = 4 ; X2 = X1 * X2 ;)
- Stockage dans la mémoire (X2 = 8 ; @addr = 1 ;)
- Réalisation d'une division (X1 = 2 ; X2 = 4 ; X2 = X2 / X1 ;)
- Stockage dans la mémoire (X2 = 2 ; @addr = 2 ;)
- Lecture de la mémoire (@addr = 0 ; X2 = 6 ;)

Voici le choronogramme obtenu (Figure **??**) :

FIGURE 9 – Opération addition, multiplication et division du processeur avec la QuadRAM

FIGURE 10 – Opération lecture de la mémoire du processeur avec la QuadRAM